qTest Reporting Gaps & Improvement Opportunities – Overview

Page is about?

Team: *QA Automation – Government Savings*
Purpose: *Identify current limitations and propose enhancements to improve visibility, reduce duplication, and align qTest reporting with automation and release needs.*

Current Gaps & Challenges

1. Limited Dashboard Flexibility

- **Dashboard (qTest Insights) & Reporting Flexibility**
    - How to **group/filter dashboards** by:
        - Release cycles
        - Automation tags (`@daily`, `@smoke`, `@feature`)
        - Application modules or plans (e.g., NY Direct, AK ABLE)
    - Using **dynamic widgets** that respond to custom fields or folder structure.

2. Missing Regression-Specific Visibility

- No out-of-the-box view to show:
    - Creating views to show:
        - Pass/fail summary per run/week/month
        - **Flaky test trend detection**
        - **Trend reports** across last 5–10 runs
    - Best practices for **cleaning up old execution data** or archiving stale reports.

3. Execution Source Tracking

- Distinguishing tests triggered via **CI (Jenkins)** vs. **manual execution**
- Best approach for consistent linkage between automation results and test runs.
- Overview of **"automation vs manual coverage"** comparison.

4. Release Readiness Reporting

- Generating **release-level health summaries** based on test execution.
- Avoiding manual tagging and Excel exports – how to streamline within qTest.

5. Automated Reporting & Notifications

- Setup of **daily automated regression reports**
- qTest **notification rules** for test failures or execution anomalies *(Prefer teams/email notification/alert)*
- Configuring recipient groups for alerts.

6. General Best Practices

- Structuring folders and test cases to **avoid duplication**
- Guidelines on **tagging strategy and modular organization**

Improvement Goals

Area	What We Need	Impact
Custom Dashboards	Drag-and-drop widgets with filters by tags/modules	Help stakeholders monitor critical tests
Regression Trends	Auto-generated trends showing test stability over time	Identify flaky vs stable tests easily
CI/CD Integration View	Visual map of Jenkins-run executions vs manual	Traceability and execution confidence
Release Coverage Snapshot	Summary of pass/fail/block by release/module	Release readiness and risk-based planning

Tech Stack: We are currently using:

- **Framework**: Java + Selenium + TestNG + Cucumber(BDD), PI -> Rest Assure
- **CI/CD**: Jenkins
- **Version Control**: GitLab

- **Database**: Oracle
- **Defect Management**: JIRA
- **Reporting & Test Management**: qTest + qTest Insights

Additional Considerations

- Nightly **regression** runs are executed via **Jenkins**, results synced to qTest.
- Test cases are tagged with `@daily`, `@smoke`, and similar keywords.
- **Release cadence is bi-weekly**, with progressive test coverage expansion.
- Aligning dashboards with this structure would streamline visibility for QA, Dev, and Product teams.